

P00: Visibilidade

Aula 05 – Visibilidade.

Prof.: Cícero Santos

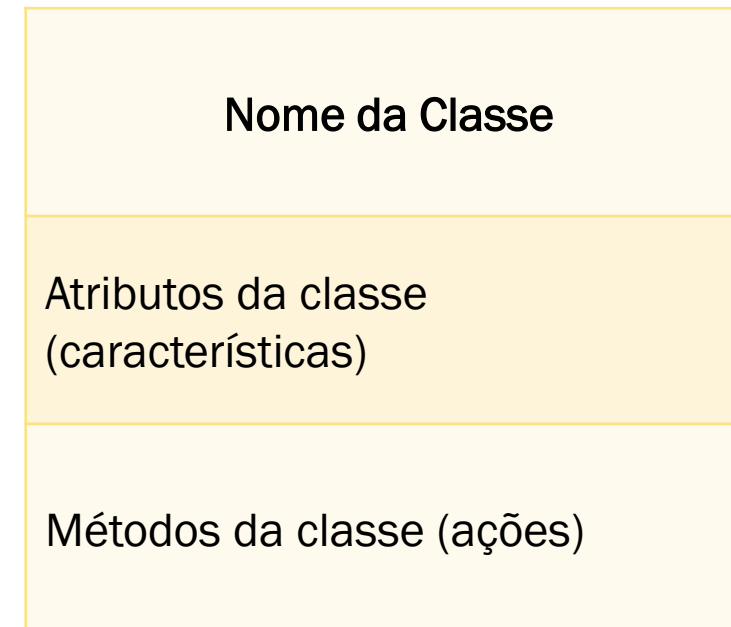
Agenda

- **P00**
 - Rever
 - UML - Conceitos
 - **Visibilidade**
 - O que é
 - Por que usar
 - E como implementar

Linguagem de Modelagem Unificada - UML

- **UML**

- Representação visual de um classe
 - Retângulo → classe
- Por que usar?
 - Representatividade de objetos físico ou abstratos.
 - Imagine como representar um 'objeto' agendamento, por exemplo.



Visibilidade - UML

- **Como representar visibilidade na UML?**
 - Representa visibilidade Privado (–)
 - Representa visibilidade Publico (+)
 - Representa visibilidade Protegido (#)

Pessoa
id + nome + email
+ falar() # enviarMensagem() -- consultarCpf()

P00: Visibilidade

- **O que é visibilidade?**
 - Indicam o nível de acesso de um componentes internos de uma classe.
- **Se aplica:**
 - Atributos
 - Métodos
 - [...]

Visibilidade Privada (-)

- **private**
 - Restringe a iteração
 - Somente a classe atual terá acesso
 - Ninguém fora da classe consegue ver ou usar o atributo

Visibilidade Privada (-)

- **Imagine um cenário: Comprador, balconista e caixa.**
- **Analogia acesso privado**
 - **Comprador:**
 - O cartão de fidelidade é um atributo privado. Apenas o comprador pode ver ou modificar as informações contidas nele. O balconista e o caixa não têm acesso a esses dados.

Visibilidade Protegido (#)

- **protected**
 - A classe atual e todas as sub-classe/classe filha terá acesso.
 - [...] todo mundo vai poder ver e usar esse atributo livremente de que seja membro da classe super/pai

Visibilidade Protegido (#)

- **Imagine um cenário: Comprador, balconista e caixa.**
- **Analogia visibilidade protegida**
 - **Balconista:**
 - O balconista pode acessar informações sobre o tipo de desconto que o comprador tem, mas apenas se ele for um funcionário da loja. Se o balconista for um amigo do comprador fora do trabalho, ele não terá acesso a essas informações.

Visibilidade Público (+)

- **public**
 - É o nível mais permissível
 - A classe atual e toda as outras classes terá acesso
 - [...] todo mundo vai poder ver e usar esse atributo livremente

Visibilidade Público (+)

- **Imagine um cenário: Comprador, balconista e caixa.**
- **Analogia visibilidade público**
 - **Caixa:**
 - O caixa pode acessar o total da compra e processar o pagamento. Essa informação é pública e deve ser acessível a qualquer parte do sistema.

Outro exemplo

Identifique os “atributos” privados, protegido e públicos neste imagem.



Resumo dos Níveis de Visibilidade



- **Privado**
 - Acesso apenas pela própria classe
- **Protegido**
 - Acesso pela própria classe e subclasses
- **Público**
 - Acesso global (aqui é festa!)

Visibilidade na prática

- **Como definir a visibilidade de um atributo:**
 - Exemplo:
 - visibilidade tipo atributo;
 - Prático:
 - `private string nome;`
- **Como defino a visibilidade de um método?**
 - Exemplo:
 - visibilidade retorno `nomeDoMetodo(parâmetros?){}`
 - Prático:
 - `public void consultarNome(string nome){}`

Exercício

Implemente essas duas classes no NetBeans com Java

Cliente

id: int
+ nome
+ renda
+ score

+ abrirConta()
+ solicitarEmprestimo(double valor)
+ consultarScore(): int
+ aumentarScore(int pontos)
+ diminuirScore(int pontos)

Conta

numero: int
+ agencia
+ saldo
+ limite
+ status
+ titular: Cliente

+ depositar(double valor)
+ sacar(double valor)
+ transferir(destino, valor): boolean
+ consultarSaldo(): double
+ bloquearConta(): boolean
+ ativarConta(): boolean



Programação Orientada a Objeto

Aula 05 – Métodos Getters, Setters e Construtor.

Prof.: Cícero Santos

Agenda da Aula

- **Hoje vamos estudar:**
 - Revisão rápida de Classe e Objeto
 - Encapsulamento na prática
 - Métodos Getters
 - Métodos Setters
 - Construtores
 - Sobrecarga de Construtores
 - Boas práticas segundo Deitel
 - Analogia completa: “Classe como Cofre Inteligente”

Objetivos

- **Ao final da aula, o aluno deverá ser capaz de:**
 - Compreender o conceito de encapsulamento
 - Criar métodos get e set corretamente
 - Entender o papel do construtor na inicialização de objetos
 - Aplicar validações em setters
 - Diferenciar acesso direto de atributos vs. acesso controlado
 - Aplicar boas práticas recomendadas por Deitel (2017)

Relembrando: Classe e Objeto

- **Relembrando:**
 - Uma classe é um modelo, uma representação de um objeto real.
 - Um objeto é uma instância da classe.
 - Analogia:
 - Classe = Planta de uma casa
 - Objeto = Casa construída
 - Atributos → características
 - Métodos → comportamentos



Encapsulamento

- Ocultar os dados da classe e permitir acesso controlado por métodos.
- Analogia do Cofre:
 - Imagine uma conta bancária:
 - O saldo não pode ser alterado livremente.
 - Você precisa de uma “interface segura”.

```
public class Conta {  
    private double saldo;  
}
```

- O atributo deve ser **private**.

Se deixarmos **public**, qualquer parte do sistema pode alterar o valor indevidamente.

Métodos Getters

- **Getter = método que retorna o valor de um atributo privado.**
 - Estrutura padrão:

```
public double getSaldo() {  
    return saldo;  
}
```
 - Analogia: Getter é como.
 - Um extrato bancário
 - Uma janela para visualizar o que está dentro do cofre
 - Um visor digital
 - Ele mostra, mas não altera.

Métodos Setters

- **Setter = método que modifica o valor de um atributo.**

```
public void setSaldo(double saldo) {  
    this.saldo = saldo;  
}
```

Métodos Setters

- **Setter = método que modifica o valor de um atributo.**
- **Analogia: Setter é como.**
 - Um caixa eletrônico
 - Um porteiro que verifica antes de permitir entrada
 - Um painel de controle
- **Aqui podemos aplicar regras de validação!**

```
public void setSaldo(double saldo){  
    this.saldo = saldo;  
}
```


Setters com Validação

- **Setters devem proteger o estado do objeto.**
 - O setter é um segurança:
 - Se valor for inválido → não entra.
 - Protege a integridade do objeto.
- **Isso mantém o objeto consistente.**

```
public void setSaldo(double saldo)
{
    if (saldo >= 0) {
        this.saldo = saldo;
    }
}
```

Construtor

- **O construtor:**
 - Tem o mesmo nome da classe
 - É executado automaticamente ao criar o objeto
 - Inicializa atributos
- **Constrói o objeto com valores iniciais/predefinido**

```
public class Conta {  
  
    private double saldo;  
  
    public Conta(double saldoInicial) {  
        saldo = saldoInicial;  
    }  
}
```

Analogia do Construtor

- **Construtor é como:**
 - O momento da construção da casa
 - A abertura da conta bancária
 - A configuração inicial de um celular novo
- **Sem construtor adequado:**
 - O objeto pode nascer em estado inválido.

Sobrecarga de Construtores

- Podemos ter vários construtores:
- Analogia:
 - É como comprar um carro:
 - Versão básica (sem opcionais)
 - Versão completa (com opcionais)

```
public Conta() {  
    saldo = 0;  
}  
  
public Conta(double saldoInicial) {  
    saldo = saldoInicial;  
}
```

Acesso Direto vs Encapsulado

- **Comparação**

Sem Encapsulamento	Com Encapsulamento
Casa sem portas	Casa com fechaduras
Sistema vulnerável	Sistema seguro
Sem controle	Com validação

Acesso Direto vs Encapsulado

- Comparação

Errado ✖

```
conta.saldo = -500;
```



Acesso Direto vs Encapsulado

- Comparação

Correto ✓

```
conta.setSaldo(500);
```



Estrutura Completa Exemplo

Observe:

- Atributos privados
- Getters públicos
- Setter com validação
- Construtor garantindo estado inicial válido

```
public class Pessoa {  
    private String nome;  
    private int idade;  
  
    public Pessoa(String nome, int idade) {  
        this.nome = nome;  
        setIdade(idade);  
    }  
    public String getNome() {  
        return nome;  
    }  
    public int getIdade() {  
        return idade;  
    }  
    public void setIdade(int idade) {  
        if (idade >= 0) {  
            this.idade = idade;  
        }  
    }  
}
```


Resumo Conceitual

- **Getter** → lê o valor
- **Setter** → modifica com controle
- **Construtor** → inicializa o objeto
- **Encapsulamento** → protege os dados

Mapa Mental da Aula

- **Classe**
 - Atributos (private, public, protected)
 - Construtor (inicializa, constrói)
 - Getters (consultam, acessam)
 - Setters (seta, modificam com validação)

Referência

DEITEL, P. J.; DEITEL, H. M. *Java: como programar*. 10. ed. São Paulo: Pearson, 2017.

- **Capítulos relacionados:**
 - Classes e Objetos
 - Encapsulamento
 - Sobrecarga de Métodos e Construtores

Obrigado

Aula 03 – Métodos Getters, Setters e Construtor.

Prof.: Cícero Santos

